Ders 28 Genişletilmiş Notu: Ana Bellek: Adres Bağlama, Takas, Bitişik Bellek Ayırma, Bölümleme ve Final Bilgilendirmesi

Ders 28 Genişletilmiş Notu: Ana Bellek: Adres Bağlama, Takas, Bitişik Bellek Ayırma, Bölümleme ve Final Bilgilendirmesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Neden Ana Bellek Yönetimi?
- 2. Bölümün Yol Haritası
- 3. Temel Donanım: Yazmaçlar, Ana Bellek ve Önbellek
- 4. Taban ve Sınır Yazmaçlarıyla Koruma
- 5. Adres Bağlama
- 6. Mantıksal Adres, Fiziksel Adres ve MMU
- 7. Dinamik Yükleme ve Dinamik Bağlama
- 8. Takas (Swapping)
- 9. Mobil Sistemlerde Takasın Yerine Kullanılanlar
- 10. Bitişik Bellek Ayırma
- 11. Dış Parçalanma ve Sıkıştırma
- 12. Bölümleme (Segmentation)
- 13. Derste İşlenmeyen Kısımlar
- Örnek Sorular
- Dersin Toparlaması

1. Neden Ana Bellek Yönetimi?

Bu ders yeni yılın ilk dersiydi ve konu ana bellek (main memory) yönetimiydi. Slaytlar Silberschatz'ın *Operating System Concepts* kitabının ana bellek bölümünden (8. bölüm) alınmıştı.

Hoca derse "ana bellekte konuşulacak ne var?" sorusuyla başladı. Cevap yine tanıdık bir yerden çıkıyor: çoklu programlama (multiprogramming). Bellekte tek bir program varken işler kolaydır. O program belleği istediği gibi kullanır. Ama biz sistem kaynaklarının sonuna kadar ve en verimli şekilde kullanılmasını istiyoruz. CPU kullanımını (CPU utilization) artırmak için bellekte aynı anda birden fazla program bulundururuz.

Bellekte birden fazla program olunca iki şey sağlanmalıdır:

- Programlar birbirinin "ayağına basmadan", yani birbirinin alanına taşmadan çalışmalıdır.
- Her biri çalıştırılabilir durumda olmalıdır.

Ana bellek bölümünün bütün konuları bu iki ihtiyaçtan doğar.

2. Bölümün Yol Haritası

Hoca bölümde konuşulacakları baştan özetledi:

- **Takas (swapping):** En temel hâliyle anlatılacak. Artıları ve eksileri konuşulacak. Hoca, takasın bugün de belli açılardan, değiştirilmiş (modified) bir biçimde kullanıldığını söyledi.
- **Bitişik bellek ayırma (contiguous memory allocation):** Programlara belleği bitişik bölgeler hâlinde dağıtmaya kalkınca ne olduğu ve bunun nasıl daha verimli yapılabileceği.
- **Bölümleme (segmentation).**
- **Sayfalama (paging):** Hoca sayfalamayı şöyle tanımladı: mantıksal ve fiziksel adres uzaylarını eşit parçalara bölüp bu parçalar arasında bir eşleme (mapping) yapmak. Ama bütün parçaların kaydını tutmak gerekir. Bu kayıt, özellikle belleğin çok bol olduğu sunucu sistemlerde verimsiz olabilir. Bunu iyileştirmek için iki ya da üç seviyeli hiyerarşik yapılar ve hash tabloları kullanılabilir. Bölümün sonunda Intel ve ARM mimarilerinde bu işlerin nasıl yürüdüğüne bakılır.

Hoca, derste ne kadar yetişirse o kadarının anlatılacağını, ama anlatılamayan kısımlardan da öğrencilerin sorumlu olduğunu, bölümün sonuna kadar bakılması gerektiğini en başta söyledi.

3. Temel Donanım: Yazmaçlar, Ana Bellek ve Önbellek

3.1 İşlemcinin doğrudan erişebildiği yerler

İşlemcinin doğrudan erişebildiği depolama alanları yazmaçlar (registers) ve ana bellektir. İkincil depolama (secondary storage) ya da yedek depolama (backing store) dediğimiz diske işlemcinin doğrudan erişimi yoktur.

Komutlar çoğu zaman ya bir yazmaç parametresi ya da bir bellek parametresi alır. Komut döngüsü getir (fetch), çöz (decode), çalıştır (execute) biçimindedir:

- Önce bellekten komut getirilir ve çözülür.
- Operand varsa ve bellekteyse, o da bellekten getirilir.
- Çıkan sonuç bazen yazmaçta, bazen yeniden bellekte saklanır.

Bütün erişim bellek birimi üzerinden yapılır. Fakat bellek birimi bu adresi kimin ürettiğini bilmez. Ona yalnız bir adres akışı gelir: bazen yalnız okumak için bir adres, bazen yazmak için adres artı veri. İşlemcinin ürettiği adresler ile bellekteki fiziksel adresler arasında bir eşleme yapılması gerekir. Bu işi yapan yapıyı birazdan bellek yönetim birimi (memory management unit, MMU) olarak göreceğiz.

3.2 Bellek erişimi ve bekleme (stall)

İşlemcinin yazmaca erişmesi kolay ve hızlıdır. Çoğunlukla bir saat çevriminde (cycle), bazen birkaç çevrimde biter. Belleğe erişmek ise veri yoluna (bus) çıkmak ve orada işlem yapmak demektir. Bu, yazmaç erişimine göre çok daha fazla çevrim gerektirir. Bu sürede işlemci kullanıcı kodu çalıştırmaz, erişimle uğraşır. Buna bekleme (stall) denir.

Hoca burada daha önce konuşulan bir konuyu hatırlattı: Modern işlemcilerde çekirdek sayısının iki katı mantıksal iş parçacığı (logical thread) görülmesinin sebebi budur. Bir çekirdeği mantıksal olarak ikiye bölüp iki ayrı iş parçacığı gibi gösterirsek, birinin bekleme süresini diğerinin işlem süresiyle örtüştürebiliriz.

3.3 Önbellek (cache)

Çoğu sistemde bellek erişimini hızlandırmak için işlemci ile ana bellek arasına daha hızlı bellek birimleri konur. Buna önbellek (cache) denir. Fikir şudur: Bir bellek adresine eriştiyseniz, büyük olasılıkla bir sonraki adımda bir sonraki adrese ya da aynı bloktaki başka bir yere erişeceksiniz. Her erişimde bilgiyi tek tek bellekten okumak yerine bloğu tek seferde önbelleğe getirirsiniz. Sonraki erişimler önbellekten karşılanır ve bekleme sayısı azalır.

Hoca önbellekle ilgili birkaç noktaya dikkat çekti:

- Önbellek işlemcinin içindedir. Yönetimini de işlemci yapar.
- Önbellek, o an CPU'da çalışan sürecin erişimlerine göre dolar ve boşalır. Süreç CPU'yu bırakınca önbellekteki bilgi anlamsızlaşabilir. Yeni süreç kendi erişimleriyle önbelleği

doldurmaya başlar. Yer kalmayınca en az yakın zamanda kullanılan (least recently used, LRU) gibi bir yöntemle bilgiler değiştirilir.
- Bağlam değişiminde (context switch) kenara alınan bir süreç çok uzun süre sonra geri gelirse, önbellekteki bilgisi büyük olasılıkla artık orada olmayacaktır. Önbellek ıskaları (cache miss) oluşur.
- Çalışan süreç de zaman zaman önbellekte olmayan yerlere erişir. Önbelleğin bir isabet oranı (hit ratio) vardır: %80, %90, %65 gibi.

4. Taban ve Sınır Yazmaçlarıyla Koruma

Bellekte çoklu programlamayı sağlamak istiyoruz. Büyük bir fiziksel belleğin farklı noktalarına süreçler yerleştirilir. Her sürecin yalnız kendisine ayrılmış bölgeye erişmesi sağlanmalıdır. Yani bir koruma (protection) mekanizması gerekir. En temel hâliyle bu, taban ve sınır yazmaçlarıyla (base and limit registers) sağlanır.

Bir süreç belleğe erişmek istediğinde bir adres üretir. Üretilen her adres işlemci tarafından doğrudan taban ve sınır yazmaçlarıyla karşılaştırılır:

- Adres taban değerinden büyük ya da eşit mi?
- Öyleyse, taban + sınır değerinden küçük mü?
- İkisi de doğruysa erişim belleğe gider.
- Herhangi biri yanlışsa işlemci bir tuzak (trap) üretir. Süreç erişmemesi gereken bir yere erişmeye çalışmıştır.

Tuzak oluşunca çekirdek kipine (kernel mode) geçilir ve ilgili servis rutinine gidilir. Servis rutini tuzağın tipine bakar ve bunun yasadışı bir bellek erişimi olduğunu görür. Bu ölümcül bir hatadır (fatal error). İşlem orada kesilir.

> **Ders sonrası not:** Hoca bir yerde “bir işlemin kendisine ayrılan memory bölgesi içinde bir yere erişmeye çalışması oldukça sıkıntılı” dedi (00:27:28). Kastedilen, bölgenin **dışına** erişmektir. Bölge içine erişim normaldir.

4.1 Slayttaki örnek

Slayttaki süreçte taban değeri 300040, sınır değeri 120900’dür. Hoca sayıları okurken “hex de bu gerçi” dedi, ama düz okudu.

- Verilen: $base = 300040$, $limit = 120900$.
- Üst uç: $base + limit = 300040 + 120900 = 420940$.
- Geçerli adresler: $300040 \leq a < 420940$, yani son geçerli adres 420939’dur.

> **Ders sonrası not:** Hoca, sürecin “420 940’a kadar olan alanda” olduğunu söyledi. Koşul $a < base + limit$ olduğu için 420940 adresinin kendisi bölgeye dahil değildir; son geçerli adres 420939’dur. Kitaptaki bu şekildeki sayılar büyük olasılıkla onluk tabandadır, onaltılık değildir.

4.2 Yazmaçları kim değiştirir?

Hoca burada bir soru sordu: Taban ve sınır yazmaçlarının değerlerini kim koyacak, nasıl yönetilecek? İşletim sistemi süreçleri farklı zamanlarda belleğin farklı yerlerine koyar. Bu yüzden bu yazmaçların değerleri dinamik olarak değişmelidir. Her süreç için değiştirilebilir olmalı, ama süreç tarafından değiştirilememelidir.

> **Hocanın vurgusu:** Bu yazmaçları yalnız ve yalnız işletim sistemi, ayrıcalıklı kipte (privileged mode) değiştirebilmelidir.

5. Adres Bağlama

5.1 Bir programın belleğe giden yolu

Süreçlerin belleğin farklı adreslerine yerleştirildiğini gördük. Peki bir süreç nereye yerleşeceğini önceden bilir mi, ya da ne zaman bilebilir? Hoca bunu program geliştirme adımlarıyla açıkladı:

- Kaynak kod derleyiciden (compiler) ya da assembler'dan geçer ve nesne koda (object code) dönüşür.
- Kullandığınız başka kodlar varsa bağlayıcı (linker) onları bağlar.
- Yükleyici (loader), çalıştırmadan önce gerekli başka kütüphaneler varsa onları da birleştirir.
- Program çalışırken başka kütüphanelere ihtiyaç duyuyorsa dinamik bağlama (dynamic linking) yapılır.

Yani yazdığımız koda aşama aşama yeni parçalar eklenir.

Bir değişkenin işleme girmesi için bellekteki adresinin bilinmesi gerekir. Ama bunu derleme aşamasında bilmek bugün pek mümkün değildir. Bu yüzden semboller kullanılır. Bu semboller, yapıya göre bağlama sırasında, yükleme sırasında ya da çalışma sırasında gerçek adreslerle doldurulur. Programımız fiziksel adres sıfırdan başlamaz, büyük olasılıkla bambaşka bir adresten başlar. Hoca, bu başlangıç adresinin eskiden statik olabildiğini, ama artık olmadığını söyledi.

Sembolleri gerçek adreslerle eşleştirme işine adres bağlama (address binding) denir.

5.2 Üç bağlama zamanı

- **Derleme zamanı (compile time):** Değişkenlerin bellekte nereye yerleşeceğini derlerken biliyorsanız, mutlak kod (absolute code) üretebilirsiniz: "bu değişken 1004'te, bu 1008'de" gibi. Hoca, birçok mikrodenetleyicide hâlâ böyle çalışıldığını söyledi.
- **Yükleme zamanı (load time):** Kod yeniden yerleştirilebilir (relocatable) olarak üretilir. Program belleğe alınırken, konduğu adrese göre bütün adresler yeniden hesaplanır.
- **Çalışma zamanı (execution time):** Bağlama, program yüklendikten sonra çalışırken yapılır. Bunun için donanım desteği gerekir; CPU'nun da bu işte işletim sistemiyle beraber çalışması lazımdır.

Çalışma zamanında bağlamaya neden ihtiyaç duyulur? Hoca takası örnek verdi. Bellekte o an boşta duran bir sürecin bölgesini alıp ikincil depolamaya koyarız. Boşalan yere başka bir süreç yerleştiririz. Sonra başka bir yer boşalır ve ilk süreci geri getiririz. Geri getirdiğimizde onu aynı adrese koymayız, bambaşka bir adrese koyarız. Sürecin çalışma süresi boyunca farklı adreslere geçebilmesi için bağlamanın çalışma zamanında yapılması gerekir. Hocaya göre bugün birçok işletim sistemi bağlamayı çalışma zamanında yapar.

Hoca dinamik bağlamayı burada kısaca da anlattı: Koda doğrudan bir adres yazmak yerine bir saplama (stub) yerleştirilir. Bu noktaya bir referans olunca saplama dediğimiz küçük kod parçası gerekli adresi hesaplar ve sizi oraya yönlendirir.

6. Mantıksal Adres, Fiziksel Adres ve MMU

Belleğin boyutu kadar bir adres grubu vardır. Buna fiziksel adres uzayı, bu adreslere fiziksel adres (physical address) denir.

Programı geliştirirken, yüklerken, hatta çalıştırırken doğrudan erişeceğimiz fiziksel adresi bilmiyoruz. Yine de işlemci çalışırken adresler üretmelidir. Bunlar göreli (relative) adreslerdir: Programın başlangıcını 0000 kabul edersiniz.

Hocanın örneği: İşlemci bu sürecin adres uzayındaki 300. adrese erişmeye çalışıyor. Ama süreç bellekte bambaşka bir yerden, mesela 15000. adresten başlıyor. Bu durumda erişim $15000 + 300 = 15300$ fiziksel adresine gitmelidir.

Yani işlemcinin ürettiği adresler, sürecin adres uzayına ait mantıksal adreslerdir. Bellek tarafında bunlar fiziksel adrese dönüştürülmelidir:

- CPU tarafındaki adrese mantıksal adres (logical address) ya da sanal adres (virtual address) denir.
- Bellek biriminin gördüğü adres fiziksel adrestir.
- İkisi arasındaki eşlemeyi yapan yapı bellek yönetim birimidir (MMU).

Bu eşleme farklı şekillerde yapılabilir. Belleğin daha verimli kullanılması için önerilmiş bitişik bellek ayırma, bölümleme ve sayfalama gibi yöntemler vardır. Hangisi kullanılıyorsa eşleme de ona göre farklı tekniklerle yapılmak zorundadır.

6.1 Yeniden yerleştirme yazmacı

Hoca en basit MMU'yu bir şekille gösterdi. İşlemcinin ürettiği mantıksal adres 346'dır. Az önceki taban yazmacının adı burada yeniden yerleştirme yazmacı (relocation register) olur, çünkü yaptığı iş süreci bellekte bir yere eşlemek, yeniden yerleştirmektir. Yapılacak en basit iş, mantıksal adresi bu yazmaçtaki değerle toplamaktır:

$$\text{fiziksel adres} = \text{mantıksal adres} + \text{yeniden yerleştirme yazmacı}$$

Slayttaki değerlerle: $346 + 14000 = 14346$.

6.2 Intel 8086 örneği

Hoca bu yapının en çok görüldüğü yerin 8086 olduğunu söyledi. Orada dört tane yeniden yerleştirme amaçlı yazmaç vardır ve hoca öğrencilere bunların hangileri olduğunu sordu. Derste birlikte sayılan yazmaçlar:

- CS (code segment)
- DS (data segment)
- SS (stack segment)
- ES (extra segment)

Bir öğrenci SI'yı da saydı. Hoca SI'nın 8086'da bulunduğunu, ama bir segment yazmacı olmadığını, başka bir şey olduğunu belirtti.

8086'da program segmentlere bölünür ve segmentlerin içinde mantıksal adresler üretilir. Bu adreslerin fiziksel adrese dönmesi için bu dört yazmaç üzerinden eşleme yapılır. MS-DOS'un 8086 üzerindeki yapısı buydu.

> **Ders sonrası not:** Altyazıda "CS, stack segment" geçiyor (00:43:14). CS, code segment yazmacıdır; stack segment yazmacı SS'dir. Altyazıdaki "BS" de (00:43:24) büyük olasılıkla DS'dir. SI, kaynak indeks (source index) yazmacıdır.
>
> **Ek bilgi (derste anlatılmadı):** 8086'da segment yazmacının değeri doğrudan toplanmaz. Fiziksel adres $16 \times \text{segment} + \text{ofset}$ ile hesaplanır ve 20 bitliktir. Yani segment değeri 4 bit sola kaydırılıp ofsetle toplanır. Bu, basit yeniden yerleştirme yazmacı fikrinin bir çeşididir.

7. Dinamik Yükleme ve Dinamik Bağlama

Hoca slaytın farklı konuları tuhaf bir şekilde bir araya getirdiğini söyleyerek belleği daha verimli kullanmanın ve bellekte boş alan yaratmanın yollarına geçti.

7.1 Dinamik yükleme

Dinamik yükleme (dynamic loading): Bir rutine ihtiyaç yoksa o rutin belleğe getirilmez. Örneğin hata durumunda çalışan rutinler olağan akışta çalışmaz. Az çalışacak ya da çalışma olasılığı düşük rutinler doğrudan belleğe kopyalanmaz, ikincil depolamada bekler. Bir referans oluşunca, bellekte değillerse belleğe getirilirler. Bunun için rutinlerin yeniden yerleştirilebilir yükleme biçiminde (relocatable load format) olması gerekir.

Hoca bunun doğrudan işletim sisteminin yaptığı bir iş olmadığını vurguladı. Uygulama geliştirme sırasında yapılması gereken bir iştir. İşletim sisteminin katkısı en fazla, sunduğu kütüphaneleri bu şekilde gerçeklemek olabilir.

7.2 Statik ve dinamik bağlama

- **Statik bağlama (static linking):** Kullanılan kütüphane yükleme aşamasında koda gömülür. Her programın içinde kütüphanenin ayrı bir kopyası bulunur.
- **Dinamik bağlama (dynamic linking):** Bağlama işlemi, programın çalışmaya başladığı, yani süreç hâline geldiği ana kadar ertelenir. Kodda küçük bir kod parçası olan saplama

(stub) bulunur. O noktaya bir referans olunca saplama ilgili rutinin adresini bulur ve rutini çalıştırır. İşletim sistemi tarafında yapılacak iş şudur: Rutin bellekteyse doğrudan çalıştırılır, değilse bir şekilde belleğe getirilir.

Bu yapıya paylaşılan kütüphane (shared library) denir. Kütüphanelerde çok verimlidir. "Paylaşılan" denmesinin sebebi, çalışma zamanında aynı kütüphanenin n tane kopyası yerine tek bir kopyasının tutulmasıdır.

Bir sorun vardır: A programı kütüphanenin birinci sürümüyle, B ikinci sürümüyle, C üçüncü sürümüyle geliştirilmiş olabilir. Sürümler arasında geriye dönük uyumluluk (backward compatibility) yoksa bellekte her sürümden birer kopya tutmanız gerekir. Hoca bunu şöyle tarttı: n uygulama için n ayrı kopya tutmak mı, 3 ya da 5 farklı sürümden birer kopya tutmak mı? Tabii ki ikincisi daha mantıklıdır.

8. Takas (Swapping)

8.1 Temel fikir

Bellekte bir uygulamanız var. Çalıştırmak istediğiniz başka bir uygulama daha var; onun için bir süreç yaratıp programın içeriğini yüklemek istiyorsunuz. Ama bellekte yer yok. En temel çözüm şudur:

- Bellekte bir süreç seçilir. Sürecin bellek görüntüsü (image) yedek depolamaya (backing store) kopyalanır (swap out).
- Açılan yere yeni süreç oluşturulur ve içeriği doldurulur.

CPU yalnız yazmaçlara ve ana belleğe doğrudan erişir. Arada önbellek vardır, ama ona belleğe erişirken dolaylı olarak ulaşılır. Bellek hiyerarşisini hatırlarsak (yazmaç, önbellek, ana bellek, ikincil depolama) takasla artık daha yavaş bir yere erişiyoruz.

Takas kullanılınca hazır kuyruğunda (ready queue) artık iki tür süreç bulunabilir: bellektekiler ve ikincil depolamadakiler. Örneğin G/Ç beklediği için kenara alınmış, G/Ç bitince hazır duruma geçmiş ama hâlâ ikincil depolamada duran süreçler. Zamanlayıcı çalıştırmak için ikincil depolamadaki bir süreci seçerse, süreç önce belleğe getirilmelidir. Bu da bağlam değişimine ek bir yükleme süresi ekler.

8.2 Takas bugün nasıl kullanılıyor?

Hocaya göre takası bu en temel hâliyle doğrudan kullanmıyoruz, ama Linux'ta da Windows'ta da değiştirilmiş sürümleri var. Bugün bellek boyutları büyüdüğü için takas ancak ihtiyaç olunca kullanılıyor: Bellek kullanımı belli bir eşiğin, mesela %80 ya da %90'ın üzerine çıkınca takas açılıyor. Yoğunluk geçip bellek kullanımı %50, %60 gibi bir eşiğin altına düşünce takas yeniden kapatılıyor.

> **Ders sonrası not:** Altyazıda "threading'i doğrudan kullanmıyoruz" geçiyor (00:53:06). Bağlamdan kastedilen "swapping'i doğrudan kullanmıyoruz"dur.
>
> **Ders sonrası not:** Eşik aşılınca açılıp eşiğin altında kapanan takas, hocanın anlattığı genel bir modeldir. Linux'ta takas alanı büyük olasılıkla sürekli etkindir; ne kadar takas yapılacağını `vm.swappiness` gibi ayarlar ve bellek baskısı belirler.

8.3 Takas neden istenmez?

> **Hocanın vurgusu:** Takas çok istediğimiz bir şey değildir.

Hem takas dışarı (swap out) hem takas içeri (swap in) yaparken veri aktarılmalıdır: bellekten ikincil depolamaya ve ikincil depolamadan belleğe. İkisi de maliyetlidir.

Hocanın örneği:

- Verilen: Sürecin bellek görüntüsü 100 MB, aktarım hızı (transfer rate) 50 MB/s.
- Takas dışarı: $100/50 = 2$ s.
- Takas içeri: yine $100/50 = 2$ s.
- Toplam: $2 + 2 = 4$ s.

Yani bu süreler gerçekten saniyeler mertebesinde olabilir.

Bugün ikincil depolama olarak bu hızdaki diskleri değil, SSD'leri kullanıyoruz. Hoca SSD'lerde 550, 800, 1000 MB/s gibi aktarım hızlarından söz edildiğini söyledi. Fakat bir püf nokta var: SSD'lerin bir yazma ömrü vardır. Okuma ömrü çok uzundur, ama yazmanın sınırı vardır. Takas açıkken ve sürekli takas yapılırken kullanıcının bilgisini saklamak için değil, sistemin çalışması için yazıp çıkarırsınız. Bu ek yazmalar diskin ömründen yer. Bu yüzden hızlansa da takas çok tercih edilen bir şey değildir.

8.4 G/Ç bekleyen süreç sorunu

Bir süreci ikincil depolamaya alırken, sürecin o sırada herhangi bir iş yapmadığından emin olmak gerekir. Teknik olarak süreç hazır kuyruğunda olmalıdır.

Sorunlu durum şudur: Süreç bir G/Ç başlatmış ve bekleme kuyruğunda (wait queue) G/Ç'nin bitmesini bekliyor. Siz bu süreci bellekten alıp kenara koydunuz. Biraz sonra G/Ç tamamlandı. Normalde aygıtın denetleyicisi (device controller) kendi bellek bölgesindeki bilgiyi sürecin bellek alanına kopyalamalıdır. Ama süreç bellekte değil. Bilgi nereye kopyalanacak? Süreç geri geldiğinde bambaşka bir yere de yerleşebilir.

İki çözüm vardır:

- G/Ç bekleyen süreci takas etmemek.
- Takas ediliyorsa, G/Ç bilgisini çekirdek alanında (kernel space) ayrı bir yerde tamponlamak. Süreç geri gelince bilgi sürecin o anki adres uzayına kopyalanır.

İkinci yolda bilgi önce aygıt denetleyicisinden çekirdeğe, sonra çekirdekten sürece, yani iki kez kopyalanır. Bu yüzden buna çift tamponlama (double buffering) denir. Bol miktarda ek yük (overhead) getirdiği için çok tercih edilmez. Takas işlemi yapılırken bu tür kısıtlar göz önüne alınmalıdır.

9. Mobil Sistemlerde Takasın Yerine Kullanılanlar

Mobil sistemlerde takas pek kullanılmaz, doğrudan desteklenmez. Sebebi, disk olarak flash bellek kullanılmasıdır:

- Flash belleğin boyutu daha sınırlıdır.
- Yazma sayısı sınırlıdır. Hoca, belli bir sayıda (milyonlarca, milyarlarca) yazmadan sonra belleğin kaybedildiğini söyledi.

Bunun yerine iOS ve Android birbirine çok benzeyen yöntemler kullanır:

- **iOS:** Sistemdeki boş bellek belli bir düzeyin altına düşünce, uygulamalardan kendilerine ayrılmış belleği bırakmalarını ister. Bu noktada salt okunur (read-only) kısım, örneğin kod alanı bırakılır. Sürece yeniden dönülünce bu alan hızlıca ikincil depolamadan belleğe kopyalanabilir. Bu yapı uygulamanın isteğine dayandığı için istenen boş alan oluşmayabilir. O zaman süreç sonlandırılır.
- **Android:** Çok benzer bir şey yapar. Ama önce o an çalışmakta olan uygulamanın durumunu (state) bir kenara yazar.

Hoca farkı kullanıcı gözüyle anlattı: Bellek azaldığında iOS'ta bir uygulama zaman zaman tamamen kapanır ve bir anda yeniden başlar. Android ise en azından uygulamanın bıraktığı yerden devam edebileceğini garanti eder.

10. Bitişik Bellek Ayırma

Hoca aradan sonra, zaman içinde kullanılmış yöntemleri sırayla incelediğimizi hatırlatarak bitişik bellek ayırmaya (contiguous allocation) geçti.

10.1 Belleğin iki kısmı

Ana bellekte işletim sistemi ve kullanıcı süreçleri bulunur. Kullanıcı süreci tek olmak zorunda değildir, birden fazla olabilir. Bunların birbirinden ayrılması gerekir: işletim sistemi kullanıcı süreçlerinden, kullanıcı süreçleri de birbirinden.

Mimarinin yapısına, yani kesme servis rutinlerinin nerede olduğuna bağlı olarak işletim sistemi belleğin alt ya da üst ucunda, kesme vektörünün (interrupt vector) hemen devamından başlatılır. Öbür uçtan kullanıcı süreçleri başlatılır. Her kullanıcı süreci bitişik tek bir bölgede tutulur.

10.2 Koruma

Her süreç ve işletim sistemi birbirinden korunmalıdır. Bunun için yine taban ve sınır yazmaçları gerekir. Çalışan bir süreç bir adres ürettiğinde, adresin kendi bellek bölgesi içinde olup olmadığı kontrol edilir. Bölge dışındaysa bir tuzak üretilir ve adresleme hatası (addressing error) olduğu bildirilir. Ayrıcalıklı kipe geçilir, işletim sistemi devreye girip hatayı bildirir ve süreç büyük olasılıkla sonlandırılır.

Slayttaki şekil bu kontrolün donanım hâlidir: İşlemcinin ürettiği mantıksal adres sınır yazmacıyla karşılaştırılır. Sınırdan küçükse yeniden yerleştirme yazmacıyla toplanır ve bellekteki yer bulunur. Değilse tuzağa düşülür.

10.3 Sabit bölümler

Süreçlerin hepsi aynı boyda olmak zorunda değildir. Bir sürece bellek ayrılırken sürecin sığabileceği bir alan seçilmelidir. Bu alana bölüm (partition) denir.

Bölümler sabit boyutlu (fixed size) olabilir. Hoca burada sordu: Sabit boyut kullanınca sizi kısıtlayan şey nedir? Bölüm boyu, sistemde olabilecek en büyük çalıştırılabilir dosyanın boyuna göre seçilmelidir. Ama her zaman o boyda bir şey çalıştırmazsınız. Sürece ayrılan alanın büyükçe bir kısmı kullanılmadan kalabilir. Bölüm boyunu büyük tutarsanız bellekte aynı anda tutabileceğiniz süreç sayısı azalır, çoklu programlama derecesi (degree of multiprogramming) düşer ve kaynağı verimli kullanmamış olursunuz.

Hoca bu statik yapının IBM'in eski OS/360'ında kullanıldığını, sonra onların da değişken bölüm boyutuna geçtiğini söyledi.

10.4 Değişken bölümler ve boşluklar

Değişken bölümde (variable partition) her sürece kullanacağı yer kadar alan ayrılır. Ama bu yapıda bile sorunlar çıkar.

Hoca slayttaki şekli soldan sağa izledi: Önce süreç 5, süreç 8 ve süreç 2 belleğe alınmış. Süreç 8 bitince yerine süreç 9 gelmiş, ardından süreç 10 gelmiş. Süreç 10 ile süreç 2 arasında bir boşluk kalmış. Belki çalışacak bir süreç var, ama boyu bu boşluğa sığmıyor ve çalıştırılamıyor.

Boşluklar tek bir noktada olmak zorunda değildir. Çok sayıda sürecin girip çıktığı büyük bir bellekte farklı noktalarda delikler oluşur. Örneğin bir adım sonra süreç 9 tamamlanıp çıksa ve sistemde süreç 8 boyunda bir süreç bekliyor olsa: Süreç 9'un boşluğu ile diğer boşluğu birleştirebilseydiniz onu çalıştırabilirdiniz. Ama ayrı noktalarda oldukları için çalıştıramazsınız.

Bu deliklere boşluk (hole) denir. Kimisi çok büyük, kimisi çok küçük olabilir. Büyüklere bir şey koyabilirsiniz, küçüklere koyamazsınız.

10.5 Boşluk seçme stratejileri

Süreç 9 çıktıktan sonra iki boşluk olsun ve yeni süreç ikisine de sığsın. Hangisini seçeceksiniz? Bunun için üç yaklaşım vardır:

- **İlk uyan (first fit):** Belleği taramaya başlarsınız. Süreci yerleştirebileceğiniz ilk boşluk neredeyse oraya yerleştirirsiniz. Hoca bir ayrıntı ekledi: Her seferinde aramaya en baştan başlamak zorunda değilsiniz. Bir istek için belleğin bir kısmını tarayıp yerleştirdiyseniz, ikinci isteği taramaya kaldığınız yerden devam edebilirsiniz. Her seferinde başa dönmek yerine dönüşümlü (rotatif) bir yaklaşım olabilir.
- **En iyi uyan (best fit):** Sürecin çalışmasına yetecek en küçük boşluk aranır. En uygun yer bulunmaya çalışılır.
- **En kötü uyan (worst fit):** En geniş boşluk bulunur ve süreç oraya yerleştirilir.

Hoca, en kötü uyanın bu adla söylenince saçma gelebileceğini, ama o kadar da saçma olmadığını söyledi. Mantık şudur: En geniş alanı ayırırsam geriye kalan boşluk da büyük olur. Boşluk büyük olursa oraya başka bir süreç yerleştirmem mümkün olur.

Hangisi daha iyi? Bu kanıtlanabilir bir durum değildir, ama simülasyonlarda görülen şudur:

- First fit ve best fit, worst fit'e göre hem hız hem bellek kullanımı açısından daha verimlidir.
- First fit ile best fit arasında doğrudan "bu ondan iyidir" denemez.
- Ama first fit'in normalde best fit'ten hızlı olması beklenir. Best fit'i bulmak için bütün listeyi taramak gerekir. First fit'te ise uygun boşluğu bulunca yerleştirirsiniz.

  Ders sonrası not: Altyazıda best fit bir yerde "Westbit" olarak geçiyor (01:22:58, "Westbit'te tam en uygun yeri aramaya çalışıyorsunuz"). Bu cümle best fit'i anlatır; hemen ardından gelen "Westbit'te ise en geniş boşluk" cümlesi worst fit'tir.

  Ek bilgi (derste anlatılmadı): Best fit'te bütün listeyi taramak, boşluk listesi boya göre sıralı tutulmuyorsa gerekir. Liste boya göre sıralıysa yetecek ilk boşluk aynı zamanda en küçük uygun boşluktur. Worst fit için de, liste sıralı değilse bütün liste taranmalıdır.

11. Dış Parçalanma ve Sıkıştırma

11.1 Dış parçalanma

First fit ve best fit iyi stratejiler olsa da onlarda da kullanılamayan boşluklar oluşur. Bir yere ayırma yaptınız ve geriye bir boşluk kaldı. Bu boşluk gelen süreçlerin boyundan küçükse kullanılamaz olarak orada kalır. Ya da art arda bir grup süreç yerleştirdiniz, aradan biri çıktı, oraya aynı boyda bir süreç bulamadınız. Bölümler arasında kullanılamayan boşluklar oluşur.

Bu küçük boşlukları bir araya getirebilseniz belki bir işlemi çalıştırabilirdiniz, ama bitişik olmadıkları için çalıştıramazsınız. Bu duruma dış parçalanma (external fragmentation) denir.

11.2 %50 kuralı

İstatistiksel analizlere göre first fit için şu sonuç vardır: N tane ayrılmış blok varsa, bunun yarısı kadar, yani $0{,}5N$ blokluk alan parçalanmaya kurban gider. Buna %50 kuralı (50-percent rule) denir.

Belleğin ne kadarının kullanılamadığını hesaplayalım (ara adım eklendi):

- Ayrılmış: N blok.
- Parçalanmaya giden: $0{,}5N$ blok.
- Toplam: $N + 0{,}5N = 1{,}5N$ blok.
- Kullanılamayan oran:

$$\frac{0{,}5N}{1{,}5N} = \frac{1}{3}$$

Yani belleğin yaklaşık üçte biri kullanılamaz hâle gelir.

> **Ders sonrası not:** Hoca formülü "n artı bir bölü iki n" diye okudu (01:27:30). Doğru hesap yukarıdaki gibidir: $0{,}5N/(N + 0{,}5N) = 1/3$.

11.3 Sıkıştırma (compaction)

Dış parçalanmayı düzeltmenin bir yolu sıkıştırmadır (compaction). Farklı noktalardaki boşlukları bitişik hâle getirmek için süreçler bir noktadan başka bir noktaya taşınır. Böylece boşluklar bir araya gelir ve daha büyük bir boşluk elde edilir.

Güzel bir çözüm gibi görünse de temel sorun adres bağlamadır. Bağlama derleme zamanında yapılıyorsa yeniden yerleştirme şansınız yoktur; sıkıştırma yapılamaz.

Ek bilgi (derste anlatılmadı): Kitaba göre sıkıştırma yalnız yeniden yerleştirme dinamikse, yani bağlama çalışma zamanında yapılıyorsa mümkündür. Yükleme zamanında bağlanmış kod da adresleri yükleme anında sabitlendiği için çalışırken taşınamaz. Ayrıca süreçleri bellekte kaydırmanın kendisi de maliyetli bir işlemdir.

11.4 Bitişiklik şartını kaldırmak

Dış parçalanmanın başka bir çözümü, şimdiye kadar hep varsaydığımız bitişiklik kısıtını kaldırmaktır. Süreç parçalara bölünür ve bu parçaların bellekte farklı noktalarda olmasına izin verilir. Böylece dış parçalanma ortadan kaldırılabilir. Bu kısıtı kaldıran tekniklerden biri bölümlemedir.

12. Bölümleme (Segmentation)

12.1 Programcının bakışı

Uygulama geliştiricinin gözünden bakınca bellek, doğrusal ilerleyen bir adres dizisi yerine farklı parçalar, farklı segmentler olarak ifade edilebilir: ana program, bir prosedür, bir fonksiyon, bir metot, bir nesne, bir yığın (stack) alanı, bir dizi. Bunlar daha mantıksal seviyede parçalardır.

Bölümlemede yapılan da budur: Program, yani süreç, farklı segmentlere bölünür. Bu segmentler bellekte farklı yerlere yerleştirilebilir.

12.2 Mantıksal adres ve segment tablosu

Segmentlere isim verilebilir, ama daha yönetilebilir olması için adresin bir parçası olarak numara verilir. Numaralanan segmentler bellekte farklı yerlere yerleştirilir. Bunun için donanım desteği gerekir.

- Mantıksal adres iki parçaya bölünür: ⟨segment numarası, ofset⟩. Örneğin “birinci segmentin bininci adresi” gibi.
- Segment numarasını bellekteki yerle eşleştirecek bir yapı gerekir: “Segment 1 dediğimde, segment 1’in başlangıç adresi burasıdır.”
- Sonra adresin ofset kısmı bu başlangıca eşlenir.

Bunları yönetmek için bir segment tablosu (segment table) gerekir. Tablo hem segmentlerin başlangıç adresini, yani tabanını (base), hem de segmentlerin boyunu, yani sınırını (limit) tutar.

Çevirme adımları:

- Mantıksal adresin segment kısmı segment tablosunda bir indise denk düşer.
- Oradan sınır ve taban okunur.
- Ofset sınırdan küçükse, yani adres segmentin sınırları içindeyse, ofset tabanla toplanır ve fiziksel adres bulunur.
- Değilse tuzağa düşülür.

12.3 Koruma ve izinler

Bellekte farklı süreçlere ait farklı segmentler bir arada bulunur. Bunlar birbirinden korunmalıdır. Ayrıca segmentlerin türüne göre farklı izinler gerekir. Daha önce 8086’da bile kod segmenti, veri segmenti gibi segmentler olduğunu konuştuk. Bazı segmentlerin yalnız okunur, bazılarının okunur-yazılır, bazılarının çalıştırılabilir (executable) olması beklenir. Örneğin veri segmentinin çalıştırılabilir olmasına gerek yoktur, ama kod segmentinin çalıştırılabilir olması gerekir.

13. Derste İşlenmeyen Kısımlar

Hoca bölümlemeden sonra sayfalamaya girip girmemeyi tarttı ve konunun epey uzayacağını söyleyerek dersi orada kesti. Ama başta söylediği geçerliydi: 8. bölümün bellek yönetimiyle ilgili kısmının sonuna kadar sorumluluk vardır.

> **Hocanın vurgusu:** Sayfalama (paging), bellek koruma (memory protection) ve paylaşılan sayfalar (shared pages) dahil, yaklaşık 60. slayta kadar olan kısma bakılmalıdır.

Bölümün başında hocanın saydığı sayfa tablosu yapıları (hiyerarşik, hash tabanlı) ile Intel ve ARM örnekleri de bu kapsamdadır ve kitaptan çalışılmalıdır.

Örnek Sorular

Bu derste kare yoktur. Sorular derste sözlü sorulanlardan alınmıştır.

Soru 1 (derste sözlü, 00:27:54): Taban ve sınır yazmaçlarının değerlerini kim koyar, kim değiştirebilir?

> **Çözüm (derste verildi):** Süreçler belleğin farklı yerlerine farklı zamanlarda yerleştirildiği için değerler dinamik olarak değişir. Her süreç için değiştirilebilir olmalı, ama süreç tarafından değiştirilememelidir. Yalnız işletim sistemi, ayrıcalıklı kipte değiştirebilir.

Soru 2 (derste sözlü, 00:41:55): 8086'da yeniden yerleştirme amacıyla kullanılan dört segment yazmacı hangileridir?

> **Çözüm (derste verildi):** CS (code segment), DS (data segment), SS (stack segment), ES (extra segment). SI bir segment yazmacı değildir.

Soru 3 (derste sözlü, 00:56:58): Bir süreç G/Ç bitmesini beklerken takas edilirse ve G/Ç tamamlanırsa, aygıttan gelen veri nereye kopyalanır?

> **Çözüm (derste verildi):** Süreç bellekte olmadığı ve geri geldiğinde başka bir yere yerleşebileceği için veri doğrudan sürecin alanına yazılamaz. Ya G/Ç bekleyen süreç takas edilmez ya da veri çekirdek alanında tamponlanıp süreç dönünce adres uzayına kopyalanır (çift tamponlama, ek yükü vardır).

Soru 4 (derste sözlü, 01:18:50): Sabit boyutlu bölüm kullanınca sizi kısıtlayan şey nedir?

> **Çözüm (derste verildi):** Bölüm boyu sistemdeki en büyük çalıştırılabilir dosyaya göre seçilmelidir. Küçük süreçler bölümün büyük kısmını boş bırakır. Bölüm büyük tutulunca bellekte aynı anda tutulabilen süreç sayısı ve çoklu programlama derecesi düşer.

Soru 5 (derste sözlü, 01:24:35): First fit'in best fit'ten neden daha hızlı olması beklenir?

> **Çözüm (derste verildi):** Best fit'te en küçük uygun boşluğu bulmak için bütün listeyi taramak gerekir. First fit'te ise yetecek ilk boşluk bulununca süreç yerleştirilir ve arama biter.

Soru 6 (derste sözlü, 00:54:24): 100 MB bellek görüntüsü olan bir süreç 50 MB/s aktarım hızıyla takas edilirse ne kadar süre harcanır?

> **Çözüm (derste verildi):** Takas dışarı $100/50 = 2$ s, takas içeri $100/50 = 2$ s, toplam 4 s.

Dersin Toparlaması

Bu derste bellek yönetiminin temeli, çoklu programlamanın getirdiği iki ihtiyaçtan kuruldu: süreçlerin bellekte bir arada bulunması ve birbirinden korunması. İşlemci yalnız yazmaçlara ve ana belleğe doğrudan erişir; bellek erişimi yavaştır ve önbellek bu farkı kapatmaya çalışır. Koruma en basit hâliyle taban ve sınır yazmaçlarıyla sağlanır ve bu yazmaçları yalnız işletim sistemi değiştirir. Sembolik adresler derleme, yükleme ya da çalışma zamanında gerçek adreslere bağlanır. Bugünkü sistemler çalışma zamanında bağlama yapar ve mantıksal adresi fiziksel adrese MMU çevirir. Dinamik yükleme ve paylaşılan kütüphanelerle bellekte yer açılır. Takas belleği genişletir ama yavaştır, diskin ömründen yer ve G/Ç bekleyen süreçlerde dikkat ister; mobil sistemler bunun yerine uygulamalardan bellek bırakmalarını ister.

Bitişik bellek ayırmada sabit bölümler belleği boşa harcar, değişken bölümler ise boşluklar ve dış parçalanma doğurur. First fit, best fit ve worst fit stratejileri arasında first fit ve best

fit daha iyidir, ama %50 kuralına göre belleğin yaklaşık üçte biri yine kullanılamaz. Sıkıştırma ancak çalışma zamanında bağlamayla mümkündür. Kalıcı çözüm bitişiklik şartını kaldırmaktır: Bölümleme bunu programcının mantıksal bakışıyla, segment tablosu üzerinden yapar. Aynı fikrin eşit boyutlu parçalarla yapılan hâli olan sayfalama derste işlenmedi, ama finale dahildir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.